

Callback functions

Brainster Web Development Academy



Table of contents

- 01 Callbacks
- 02 Exercise 1
- 03 Functional iteration
- 04 Exercises
- 05 Event callbacks
- 06 Exercises



Callbacks

- Since variables can be functions, we can also pass them as arguments to other functions;
- That means, we can also invoke other functions from within functions;
- Functions that are passed as arguments and called from within a function are also known as “**callbacks**”.



Callbacks

```
function ping() { console.log('ping'); }  
function pong() { console.log('pong'); }  
function pingPong(callback) {  
    ping(); // call one function from the outside  
    pong(); // call another one from the outside  
    callback(); // do the callback at last (sometimes with a return)  
}  
  
// This is our sample callback  
function myCallback() {  
    console.log('This is the callback function');  
}  
  
// This is the callback function being passed as an argument  
pingPong(myCallback); // ping; pong;
```



Exercise 1

1. Define a function that takes two arguments - a number and a callback function.
2. Count down from the given number to 0, and then execute the callback function.
3. Define a second function that simply logs “done” to the console.
4. Prompt for a number. Parse the input to an actual number.
5. Invoke the function at 1 using the number from 4 and the second function from 3.



Functional iteration

One additional way to iterate over an array is using its built-in **.forEach()** method.

```
1 let myArr = [1, 2, 3];  
  myArr.forEach((num, index) => { // you can also use a regular function  
    console.log("index:", index, "member:", num);  
  });
```

This isn't too different than a regular for - we just don't need to know the length ahead of time to iterate over it.



Exercise 2

1. Define a function which:
 - 1.1 Takes an array of numbers as an input argument
 - 1.2 Finds the arithmetic mean of the array (use **forEach** to iterate)
 - 1.3 Writes the result in the document
2. Invoke the function at 1 using an array of numbers (**e.g. [5,8,20,55,190]**)

(**Tip:** an arithmetic mean is simply the sum of all the array numbers divided by the length of the array)



Some examples

With the Array type as an example, there are multiple built-in methods that use callbacks to solve a certain problem:

- **Array.map** - iterate over an array, and produce a new array using a callback on each member of the original array;
- **Array.find** - find a member of an array based on a condition given in a callback
- **Array.filter** - find multiple members of an array based on a condition given in a callback



Check the array reference on MDN for more info.

Exercise 3

1. Given is this array: **let arr = [1,2,3,4];**
2. Iterate over the array and create a **new array** which has every item from the first one added up by 10.
3. You should get the following in console: **[11, 12, 13, 14]**

(**Hint:** use iterator array.map)



Exercise 4

1. Given is this array: **let arr = [1,2,3,4];**
2. Iterate over the array and create a **new array** which has every even number from the first one.
3. You should get the following in console: **[2, 4]**

3

(**Hint:** use iterator `array.filter`)



Exercise 5

1. Given is this array: **let arr = [1,2,3,4];**
2. Iterate over the array and find the **first** odd number from the first one.
3. You should get the following in console: **1**

(**Hint:** use iterator array.find)



Exercise 6

3

1. Define an array of numbers of your choice.
2. Prompt for a number
3. Define a function that takes 2 parameters - the array and the number coming from prompt.
4. Inside the function, iterate through the initial array and return a **new** array in which every item from the initial array is multiplied with the number that comes from prompt.
5. **Filter** the new array where you put only numbers that are bigger or equal than 20.
6. Return the array from the function.
7. Example:
arr1 = [1,2,3,4,5,6,7]; & entered number **10** in prompt

the result in console should be: **[20, 30, 40, 50, 60, 70]**



Event callbacks

One of the most common uses for callbacks is making things react on clicks. Namely:

```
let myButton = document.querySelector('button');  
myButton.addEventListener('click', function() {  
    console.log('I was clicked!');  
});
```

There are many more events in JS, but more on those next week ;)



Exercise 7

3

1. Define a function which:
 - 1.1 Takes three parameters: length, width, and a callback function.
 - 1.2 The functions should check if both length and width are positive numbers.
 - 1.3 If either length or width is not a positive number, call the callback function with an error message.
 - 1.4 If both length and width are positive numbers, calculate the area (length * width) and call the callback function with the result.
2. Define a JavaScript function that displays the result and which:
 - 2.1 Takes two parameters: error and result.
 - 2.2 If error is not null, display the error message on the DOM.
 - 2.3 If error is null, display the calculated area on the webpage.

